Back Propagation Algorithm

* Deriving Eq for updating weights in BPAlgorithm.
* we use gradient descent rule
* d – is a training example
  D – the complete dataset

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

E_d is error on training data eg. $E_d = \frac{1}{2} \sum_{k \in D} (t_k - o_k)^2$

$$\Delta w_{ji} = -\eta \boxed{\frac{\partial E_d}{\partial w_{ji}}} \leftarrow \text{compute}$$

using chain Rule

w_{ji} influences rest of n/w only through net_j.

$net_j \rightarrow$ weighted Sumg inputs for unit j

$$net_j = \sum_i w_{ji} x_{ji}$$

$$\frac{\partial net_j}{\partial w_{ji}} = x_{ji}$$

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \times \frac{\partial net_j}{\partial w_{ji}}$$

$$= \frac{\partial E_d}{\partial net_j} \times x_{ji}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} \times x_{ji}$$

we know $\rightarrow \eta$ learning rate

$x_{ji} \rightarrow$ input

so derive $\frac{\partial E_d}{\partial net_j}$

- if j is output unit
- if j is internal unit

Case 1: If j is output unit

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \times \frac{\partial o_j}{\partial net_j}$$

$$\boxed{\frac{\partial E}{\partial o_j}} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

$\forall k = j$

$$\therefore \quad \frac{\partial E}{\partial o_j} = \frac{1}{2} \times 2 (t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j}$$

$$= -(t_j - o_j)$$

[t_j - constant, $\partial o_j / \partial o_j = 1$]

$$\frac{\partial E}{\partial net_j} = -(t_j - o_j) o_j (1 - o_j)$$

$$\Delta w_{ji} = -\eta \left(-(t_j - o_j) o_j (1 - o_j)\right) x_{ji}$$

$$= \eta \underbrace{(t_j - o_j) o_j (1 - o_j)}_{\delta_j} x_{ji}$$

$$\boxed{\frac{\partial o_j}{\partial net_j}} = \frac{\partial \sigma(net_j)}{\partial net_j}$$

$$= \sigma(net_j)(1 - net_j) \quad \text{[according to sigmoid rule]}$$

$$= o_j (1 - o_j)$$

$$\boxed{\Delta w_{ji} = \eta \delta_j x_{ji}}$$

Case 2: if j is internal node

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in \text{Downstream}} \boxed{\frac{\partial E_d}{\partial net_k}} \times \frac{\partial net_k}{\partial net_j} \quad \text{chain rule}$$

$$= \sum_{k \in \text{downstream}} -\delta_k \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in \text{Downstream}} -\delta_k \boxed{\frac{\partial net_k}{\partial o_j}} \times \frac{\partial o_j}{\partial net_j} \quad \text{from above derivations}$$

$$= \sum_{k \in \text{Downstream}} -\delta_k w_{kj} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in \text{Downstream}} -\delta_k w_{kj} o_j (1 - o_j) \sum_{k \in D \ldots} -\delta_k \cdot w_{kj} \delta_k$$

$$\Delta w_j = +\eta \sum_{k \in D} \left(\delta_k \cdot w_{kj} \cdot o_j (1 - o_j)\right) x_{ji}$$

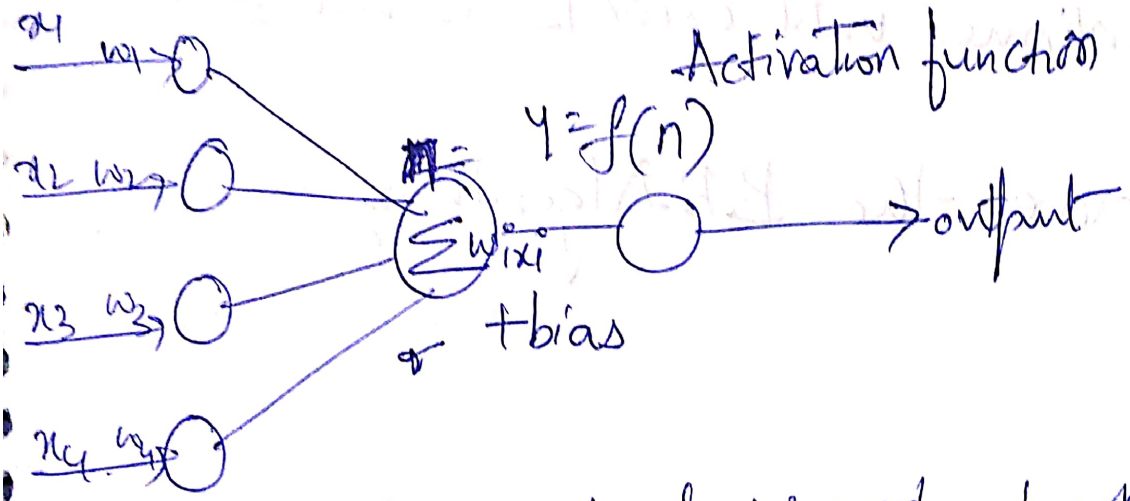
$$= +\eta \left[o_j (1 - o_j) \sum_{k \in D.} \delta_k w_{ji}\right] x_{ji} = \eta \delta_j x_{ji}$$

Backpropagation Algorithm for Feed Forward N/W.

→ BP is a supervised learning algorithm used to train feed forward neural n/w's by reducing the error between actual and predicted o/p.

→ It works in 2 phases.

① Feed Forward phase
② Back Propagation (Error correction) phase.



3-Layer feed forward network.

① I/p Layer
② one/more Layer
③ O/p Layer.

Phase-1

- Input to hidden Layer
- Hidden to output Layer
- Error Calculation.

Phase-2

Backpropagation of Error
- Error at o/p.
- Weight update & Bias

complete Backpropagation Alg

1. Initialize all weights and bias randomly
2. Provide training input vector X
3. Perform feed forward computation
4. compute output error
5. propagate error backward
6. Update weights & bias
7. Repeat for all training samples
8. Continue until error is minimized.

Derive the B.P. Algorithm